

Exam rank3 practice

▼ Bracket - 1

Verifica se uma string contém parênteses, colchetes e chaves corretamente balanceados, garantindo que cada abertura tenha um fechamento correspondente na ordem correta.

```
def bracket_validator(s: str) -> bool:
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'
    for char in s:
        if char in "([{":
            stack.append(char)
        elif char in ")]}":
            if not stack or stack.pop() != pairs[char]:
                return False
    return len(stack) == 0
```

▼ Sort list - 1

Ordena uma lista de strings usando múltiplos critérios:

1. Tamanho da string (crescente)
2. Ordem alfabética (case-insensitive)
3. Quantidade de vogais (crescente), usada como critério de desempate final

Ignora diferenças entre maiúsculas e minúsculas na ordenação principal.

```
def sort_string(str_list: list[str]) -> list[str]:
    vogal = "aAeEiIoOuU"
    def key_func(s):
        length = len(s)
        vogal_count = sum(1 for c in s if c in vogal)
    return sorted(str_list, key=key_func)
```

```
        return (vogal_count, length, s.lower())
    return sorted(str_list, key=key_func)
```

▼ Palindrome - 2

Verifica se uma string é um palíndromo.

(uma palavra, frase ou número que permanece igual quando lido da esquerda para a direita ou da direita para a esquerda, ignorando espaços, pontuação e diferenças entre maiúsculas e minúsculas)

```
def is_palindrome(s: str) -> bool:
    text = ''.join(char.lower() for char in s if char.isalnum())
    return text == text[::-1]
```

▼ Base converter - 3

Converte um número representado como string de uma base numérica de origem para uma base numérica de destino (ex: binário, decimal, hexadecimal), retornando "ERROR" para entradas inválidas.

```
def number_base_converter(num: str, src_base: int, dest_base: int) -> str:

    digits = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"

    try:
        n = int(num, src_base)

        if n == 0:
            return "0"

        result = ""
        while n > 0:
```

```

        result = digits[n%dest_base] + result
        n //= dest_base

    except Exception:
        return "ERROR"

    return result

```

▼ Pattern - 3

Conta quantas vezes aparecem pares consecutivos de dígitos em uma string

onde o segundo dígito é maior que o anterior.

A função percorre a string e verifica apenas sequências de números, ignorando outros caracteres.

```

def pattern_tracker(text: str) -> int:
    count = 0
    for i in range(len(text) - 1):
        if text[i].isdigit() and text[i+1].isdigit() and
text[i+1] > text[i] and int(s[i+1]) == int(s[i] + 1):
            count += 1
    return count

```

▼ Mirror matrix - 3

Inverte a ordem dos elementos de cada linha de uma matriz.

A função percorre a matriz e retorna uma nova matriz com cada linha em ordem reversa.

```

def reverseMatrix(s: list[list[int]]) -> list[list[int]]:
    return [e[::-1] for e in s]

```

▼ Inter - 3

Retorna uma string com os caracteres que aparecem em ambas as strings, sem repetições.

Os caracteres são adicionados na ordem em que aparecem na primeira string.

```
def inter(s1, s2):  
    res = ""  
    for char in s1:  
        if char in s2 and char not in res:  
            res += char  
    return res
```

▼ Anagram - 4

Verifica se duas strings são anagramas (se contêm exatamente as mesmas letras), com a mesma quantidade, independentemente de maiúsculas/minúsculas e espaços.

```
def Anagram(s: str, t: str) -> bool:  
  
    new_s = s.upper().strip()  
    new_t = t.upper().strip()  
  
    if len(new_s) != len(new_t):  
        return False  
  
    return sorted(new_s) == sorted(new_t)
```

▼ Shadow merge - 4

Faz a junção de duas listas de inteiros e retorna uma nova lista ordenada.

Se alguma das listas for None ou vazia, trata como lista vazia.

Após a junção, os elementos são ordenados em ordem crescente.

```
def shadow_merge(list1: list[int], list2: list[int]) -> list[int]:
    list1 = list1 or []
    list2 = list2 or []

    mixed = list1+list2
    mixed.sort()
    return mixed
```

▼ String sculptor - 5

Modifica uma string alternando letras entre minúsculas e maiúsculas.

Após cada espaço, o padrão é reiniciado.

Caracteres não alfabéticos são mantidos sem alteração.

```
def string_sculptor(text: str) -> str:
    res = ""
    lower = 1

    for c in text:

        if c.isalpha() and lower:
            res += c.lower()
            lower = 0

        elif c.isalpha():
            res += c.upper()
            lower = 1

        elif c.isspace():
            res += c
            lower = 1

        else:
            res += c

    return res
```

▼ Twister sequence - 5

Rotaciona uma lista de elementos para a direita em n posições.

Os últimos elementos da lista são movidos para o início, mantendo a ordem relativa.

```
def twister(nums, n):  
    if n < 0:  
        cut = -n  
    else:  
        cut = len(nums) - n  
        list1 = nums[cut:]  
        list2 = nums[:cut]  
        return list1 + list2
```

▼ Whisper - 6

(rotone vibes)

Se for letra (a-z ou A-Z) → desloca no alfabeto por shift

Se passar de z ou Z → "volta pro começo" (isso é o % 26)

Se não for letra → mantém igual

```
def whisper_cipher(s: str, n: int):  
    shift = ""  
    for i in s:  
        if 'a' <= i <= 'z':  
            shift += chr((ord(i) - ord('a') + n) % 26 +  
ord('a'))  
        elif 'A' <= i <= 'Z':  
            shift += chr((ord(i) - ord('A') + n) % 26 +  
ord('A'))  
        else:  
            shift += i  
    return shift
```

▼ Hidenp

Verifica se a string 'small' é uma subsequência da string 'big'.

Uma subsequência significa que todos os caracteres de 'small' aparecem em 'big' na mesma ordem, mas não precisam estar necessariamente juntos.

```
def hidenp(small, big):  
    it = iter(big)  
    return all(c in it for c in small)
```